

MOSS: 时序电路的多模态表示学习

王明君^{1,2,3,4}, 孙斌^{1,3}, 穆建安^{*1, 3}, 谷峰^{1,2,3,4}, 韩伯宇⁵, 杨天梦⁶, 张新宇^{1,3}, 刘思林^{1,3}, 温一涵⁷, 王辉⁴, 高军⁴, 赵志腾^{1,3,4}, 韩虎生^{1,3}, 子珍刘^{1,3}, 梁胜文^{1,3}, 叶静^{1,3,4}, 于北², 李小伟^{1,3}, 李华为^{*1,3}

¹中国科学院计算技术研究所处理器国家重点实验室 ²香港中文大学。 ³中国科学院大学 ⁴CASTEST有限公司 ⁵斯坦福大学 ⁶北京大学 ⁷北京工业大学

摘要：深度学习显着推进了电子设计自动化（EDA），电路表示学习成为对电路结构和功能之间关系进行建模的关键领域。现有方法主要使用大型语言模型（LLM）进行寄存器传输级（RTL）代码分析，或使用图神经网络（GNN）进行网表建模。虽然LLM擅长高层次的功能理解，但他们在详细的网表行为方面遇到了困难。然而，由于长程信息依赖性和功能监督不足，GNN 在扩展到更大的时序电路时面临挑战，导致准确性下降和泛化能力有限。为了应对这些挑战，我们提出了 MOSS，这是一种将 GNN 与 LLM 集成以进行时序电路建模的多模态框架。通过使用 RTL 代码上经过微调的 LLM 的嵌入来增强 D 型触发器（DFF）节点功能，我们将 GNN 集中在关键锚点上，减少对远程依赖性的依赖。法学硕士还提供全局电路嵌入，为功能相关任务提供有效的监督。此外，MOSS 在 GNN 中引入了自适应聚合方法和两阶段传播机制，以更好地对电路内的信号传播和顺序反馈进行建模。实验结果表明，与现有方法相比，MOSS 显着提高了时序电路的功能和性能预测的准确性，特别是在以前的模型难以实现的较大电路中。具体来说，MOSS 的到达时间预测准确率达到 95.2%。

一、简介

深度学习 (DL) 技术在 EDA 中的应用引起了广泛关注 [1], [2], , 从而在路由 [3], [4], 合成 [5], [6],和测试 [7],[8]等领域取得了显着进步。在这些深度学习驱动的 EDA 创新中，电路表示学习 [9], [10] 已成为一种有前景的研究范式，可推动后续 EDA 任务中的预测。为了进一步推进电路表示学习的发展，对时序电路进行建模尤为重要，因为它们构成了实际电子设计中最常用的电路模块。

现有的电路表示学习工作主要可分为两类，每一类都对电路描述的不同模式进行建模。第一类侧重于寄存器传输级（RTL）代码——一种抽象描述电路行为的形式语言——利用大语言模型（LLM）来提供对功能 [11]的高级理解。然而，在分析网表 [12]中各个单元的特定行为和连接性时，这种方法面临挑战。第二类采用图神经网络（GNN）进行端到端学习，对电路功能和结构 [13]之间的关系进行建模。与第一种方法相比，GNN 更接近电路结构，可以更准确地表示电路细节。相应地，基于 GNN 的模型对 1000 个门内的时序电路的切换率实现了大约 80% 的预测精度。这种细粒度的任务对于语言模型来说似乎具有挑战性。因此，使用 GNN 对电路结构进行建模以实现电路表示似乎是一种很有前途的方法。

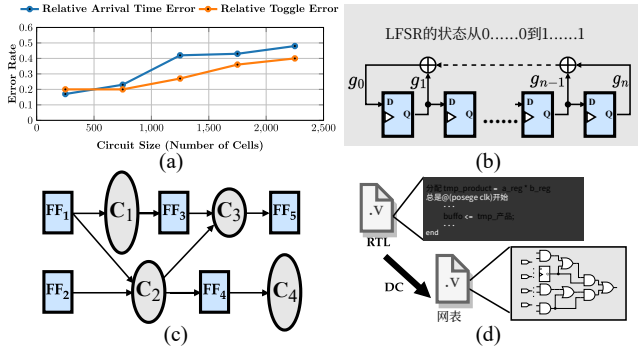


图 1 时序电路表示学习的挑战和动机。(a) 挑战1：复杂电路预测任务中现有方法的性能下降。这两个指标都是通过将每个节点的错误率与真实值相加，然后对所有节点求平均值来计算的。(b) 挑战 2：时序电路可以有几乎无限的真值表。(c) 动机 1：DFF 对时序电路进行划分。(d) 动机 2：不同的模式提供了不同层次的结构和功能抽象。

不幸的是，当使用 GNN 对更大的顺序电路进行建模以预测其功能和性能时，我们遇到了重大挑战。第一个挑战是处理远程信息依赖性的困难，这导致较大电路的准确性下降。我们实现了 GNN 模型，采用异步更新和 [14], 中的其他技术，并对不同大小的电路进行了测试，以预测切换率和到达时间。图 1（a）所示的结果表明，当应用于较大的时序电路时，现有方法的错误率显着增加。例如，在具有2000个门的电路中，预测错误率超过 40%。这种下降的一个主要原因是，与较小的电路相比，对较大的电路进行建模需要从更远的节点学习信息。尽管采用异步更新策略逐层传播远程信息，但这些先进的 GNN 设计仍然容易丢失远程信息，从而导致准确性下降。

第二个挑战是时序电路的功能难以形成有效的训练监督，导致功能相关任务的能力较弱。由于其状态特性，时序电路表现出复杂的功能行为，这使得使用 GNN 有效捕获功能变得具有挑战性。最新的工作，DeepSeq2 [14], 人为地压缩各个节点的真值表以形成训练监督。然而，这种方法有几个缺点：大规模电路效率低下：如图 1(b) 所示，由于随着时间的推移有无数种可能的状态和输入序列，时序电路可能具有几乎无限的真值表。这使得压缩真值表的方法效率低下。整体电路功能描述不充分：

MOSS: Multi-Modal Representation Learning on Sequential Circuits

Mingjun Wang^{1,2,3,4}, Bin Sun^{1,3}, Jianan Mu^{*1,3}, Feng Gu^{1,2,3,4}, Boyu Han⁵, Tianmeng Yang⁶, Xinyu Zhang^{1,3}, Silin Liu^{1,3}, Yihan Wen⁷, Hui Wang⁴, Jun Gao⁴, Zhiteng Chao^{1,3,4}, Husheng Han^{1,3}, Zizhen Liu^{1,3}, Shengwen Liang^{1,3}, Jing Ye^{1,3,4}, Bei Yu², Xiaowei Li^{1,3}, Huawei Li^{*1,3}

¹State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences ²The Chinese University of Hong Kong. ³University of Chinese Academy of Sciences ⁴CASTEST Co., Ltd. ⁵Stanford University ⁶Peking University ⁷Beijing University of Technology

Abstract—Deep learning has significantly advanced Electronic Design Automation (EDA), with circuit representation learning emerging as a key area for modeling the relationship between a circuit’s structure and functionality. Existing methods primarily use either Large Language Models (LLMs) for Register Transfer Level (RTL) code analysis or Graph Neural Networks (GNNs) for netlist modeling. While LLMs excel at high-level functional understanding, they struggle with detailed netlist behavior. GNNs, however, face challenges when scaling to larger sequential circuits due to long-range information dependencies and insufficient functional supervision, leading to decreased accuracy and limited generalization. To address these challenges, we propose MOSS, a multimodal framework that integrates GNNs with LLMs for sequential circuit modeling. By enhancing D-type Flip-Flop (DFF) node features with embeddings from fine-tuned LLMs on RTL code, we focus the GNN on critical anchor points, reducing reliance on long-range dependencies. The LLM also provides global circuit embeddings, offering efficient supervision for functionality-related tasks. Additionally, MOSS introduces an adaptive aggregation method and a two-phase propagation mechanism in the GNN to better model signal propagation and sequential feedback within the circuit. Experimental results demonstrate that MOSS significantly improves the accuracy of functionality and performance predictions for sequential circuits compared to existing methods, particularly in larger circuits where previous models struggle. Specifically, MOSS achieves a 95.2% accuracy in arrival time prediction.

I. INTRODUCTION

The application of deep learning (DL) techniques in EDA has garnered significant attention [1], [2], leading to notable advancements in areas such as routing [3], [4], synthesis [5], [6], and testing [7], [8]. Among these DL-driven EDA innovations, circuit representation learning [9], [10] has emerged as a promising research paradigm that propels predictions in subsequent EDA tasks. To further advance the development of circuit representation learning, modeling sequential circuits is particularly crucial, as they constitute the most commonly used circuit modules in practical electronic designs.

Existing works on circuit representation learning can mainly be divided into two categories, each modeling different modalities of circuit descriptions. The first category focuses on the register transfer level (RTL) code—a formal language that abstractly describes circuit behavior—utilizing large language models (LLMs) to provide a high-level understanding of functionality [11]. However, this approach faces challenges when it comes to analyzing the specific behavior and connectivity of individual cells in the netlist [12]. The second category employs graph neural networks (GNNs) for end-to-end learning, modeling the relationship between circuit functionality and structure [13]. Compared to the first approach, GNNs are closer to circuit structures, allowing for a more accurate representation of circuit details. Correspondingly, GNN-based models have achieved approximately 80% prediction accuracy for the toggle rates of sequential circuits within 1,000 gates. Such fine-grained tasks appear to be challenging for language models. Therefore, it seems to be a promising approach to use GNNs to model circuit structures for circuit representation.

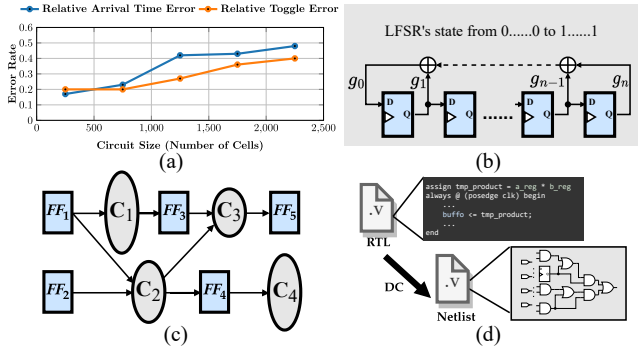


Fig. 1 Challenges and motivations for representation learning on sequential circuits. (a) Challenge 1: Performance degradation of existing methods in complex circuit prediction tasks. Both metrics are computed by summing the error ratio to the true value for each node and then averaging over all nodes. (b) Challenge 2: Sequential circuits can have almost infinite truth tables. (c) Motivation 1: DFFs divide sequential circuits. (d) Motivation 2: Different modalities provide different levels of abstraction in structure and functionality.

Unfortunately, when using GNNs to model larger sequential circuits for predicting their functionality and performance, we encounter significant challenges. The first challenge is the difficulty in handling long-range information dependencies, which leads to decreased accuracy on larger circuits. We implemented a GNN model, adopting asynchronous updates and other techniques from [14], and conducted tests on circuits of varying sizes to predict toggle rates and arrival times. The results, shown in Fig. 1(a), indicate that existing methods exhibit a substantial increase in error rates when applied to larger sequential circuits. For example, in a circuit with 2,000 gates, the prediction error rate exceeds 40%. One primary reason for this decline is that modeling larger circuits requires learning information from more distant nodes compared to smaller circuits. Despite employing asynchronous update strategies to propagate long-range information layer by layer, these advanced GNN designs remain susceptible to the loss of distant information, leading to decreased accuracy.

The second challenge is the difficulty in forming effective training supervision for the functionality of sequential circuits, resulting in weak capability in functionality-related tasks. Sequential circuits exhibit complex functional behaviors due to their stateful nature, making it challenging to capture functionality effectively with GNNs. The latest work, DeepSeq2 [14], artificially compresses the truth tables of individual nodes to form training supervision. However, this method has several drawbacks: Inefficiency for Large-Scale Circuits: As shown in Fig. 1(b), sequential circuits can have almost infinite truth tables because of the myriad possible states and input sequences over time. This makes the approach of compressing truth tables inefficient. Inadequate Description of Overall Circuit Functionality:

从每个单元的真值表中手动提取特征进行训练可能有助于比较各个节点，但形成大规模时序电路的完整功能描述具有挑战性。这种方法限制了预测完整电路行为的能力并限制了模型的泛化能力。因此，这种限制限制了需要了解电路全局功能的下游任务的性能。

有趣的是，可以利用时序电路的结构特征来减少对较大电路进行建模时遇到的远程依赖性。如图 1 (c) 所示，时序电路本质上包含某些“锚点”，特别是 D 型触发器 (DFF)。这些锚点聚合上游输入信息并将其传播到下游，从而驱动电路的功能状态。此外，DFF 之间的互连决定了到达时间等性能指标。因此，通过逐步准确地对每个 DFF 及其直接连接网络进行建模，我们可以在较短范围的依赖关系中定位关键信息。这种方法减少了对远程信息传播的依赖。然而，现有的方法尚未利用这一方面进行有针对性的建模。

另一方面，与网表中存在的大量单元级节点相比，RTL 代码提供了对电路连接及其形成的功能更宏观的描述。如图 1(d) 所示，网表中的复杂连接结构可以通过一行 RTL 代码简洁地表示，从而提供电路功能的清晰概述。因此，如果我们能够同时学习 RTL 代码和电路网表结构，协同集成局部和全局信息，将显着提高电路表示学习的效率。此外，通过利用 RTL 代码的形式语言（它提供了功能的有效抽象）和对电路数据进行微调来解释 RTL 的 LLM，我们可以生成更准确的功能描述。

在本文中，我们提出了 MOSS，一种将 GNN 与 LLM 相结合的多模态框架。我们对 RTL 代码上的 LLM 进行微调，以生成捕获全局电路功能的嵌入。这些嵌入增强了网表中 DFF 单元的功能，使 GNN 能够专注于关键锚点并减少对远程依赖性的依赖。法学硕士还提供全局功能嵌入，为功能相关任务提供有效的监督。此外，我们引入了自适应聚合方法和两级传播机制，以更好地模拟电路内的信号传播和顺序反馈。通过将 GNN 的结构见解与 LLM 的功能理解相结合，MOSS 提高了时序电路的功能和性能预测的准确性，特别是在现有方法难以解决的较大电路中。

本文的贡献总结如下：

- 我们提出了 MOSS，这是一种多模态框架，它将 LLM 与 GNN 结合起来进行顺序电路建模，增强了局部节点嵌入与电路功能和结构的全局表示之间的协作。
- 我们在 GNN 中引入了自适应聚合方法和两阶段传播机制，以更好地对电路内的信号传播和顺序反馈进行建模。
- MOSS 显着提高了时序电路的功能和性能预测的准确性，在较大电路上的性能优于现有方法。例如，它在到达时间预测方面达到了 95.2% 的准确率。

二. 背景及相关工作

A. 用于 EDA 任务的图神经网络

图神经网络 (GNN) 是一类专门用于处理图结构数据 [15] 的深度学习模型。通过直接在图上操作，它们捕获节点之间复杂的依赖关系和交互。GNN 迭代地聚合来自相邻节点的信息以更新节点表示 [16]，使其能够捕获局部和全局结构特征 [17]。在 EDA 领域，GNN 由于其对电路结构 [18], [19] 的高度适应性而被广泛用于电路表示学习、性能预测和故障检测等任务。

例如，FGNN2 [20] 和 DeepGate2 [21] 专注于组合电路，缺乏时序电路固有的时序依赖性。DeepSeq 系列 [14], [22] 试图通过逻辑训练和翻转概率来建模时序行为，从而将时间信息纳入 GNN。DeepSeq2 通过引入解缠结的结构、功能和时序表示来推进这种方法。然而，如图 1 所示，DeepSeq 系列在长期信息依赖性方面遇到了困难。当应用于更大规模的电路时，这个缺点导致精度大幅下降。此外，DeepSeq 系列在与逆变器图 (AIG) 上进行学习，与由标准单元组成的真实网表相比，其节点更简单且功能统一。在 EDA 场景中，时序等关键标签是在标准单元级别而不是 AIG 级别进行注释的。因此，基于 AIG 的模型无法直接捕获这些指标，从而限制了工业适用性。相比之下，我们的工作对由标准单元组成的网表进行建模。

B. EDA 的大型语言模型和微调任务
大型语言模型 (LLM) 在各种 EDA 任务中显示出了前景，例如设计流程自动化、错误检测和硬件生成 [23]。法学硕士经过大量文本数据的预训练，擅长理解 RTL 代码和描述 [24]。诸如低秩适应 (LoRA) 之类的微调技术可以使 LLM 有效适应 EDA 特定任务 [25]。然而，虽然 LLM 可以从 RTL 代码中捕获功能语义，但它们对电路中固有的复杂结构依赖性进行建模的能力是有限的，因为它们最初是为处理文本数据 [26] 而设计的。因此，仅法学硕士不足以完成需要功能理解和详细结构表示的电路设计任务。

C. 多模态对齐和对比学习
多模态对齐和对比学习已成功用于集成不同的数据类型 [27]，例如文本和图像 [28]，方法是在共享嵌入空间 [29] 中对齐它们的表示。像 CLIP [30] 这样的著名模型展示了对比学习在跨模式任务（包括检索和分类）中的潜力。这些方法利用不同模式的优势来提高表示学习 [31] 的性能。

三. 问题表述

时序电路建模为有向图 $G = (V, E)$ ，其中 V 表示电路组件（例如逻辑门、DFF）， E 表示组件连接，由于状态元素引入的时间依赖性，因此提出了独特的挑战。关键问题是学习编码结构和时间特征的节点嵌入 $\mathbf{H} \in \mathbb{R}^{|V| \times d}$ 。这些嵌入应该支持下游任务，例如切换率预测、到达时间预测和电路分类：

$$\mathbf{H} = f(G, \mathbf{X}, \mathbf{T}) = f(V, E, \mathbf{X}, \mathbf{T}), \quad (1)$$

Manually extracting features from truth tables on each cell for training may aid in comparing individual nodes, but it is challenging to form a complete functional description of large-scale sequential circuits. This approach limits the ability to predict the complete circuit behavior and restricts the model’s generalization capability. Consequently, this constraint limits performance on downstream tasks that require understanding the circuit’s global functionality.

Interestingly, the structural characteristics of sequential circuits can be exploited to reduce the long-range dependencies encountered when modeling larger circuits. As shown in Fig. 1(c), sequential circuits inherently contain certain “anchor points”, specifically D-type Flip-Flops (DFFs). These anchor points aggregate upstream input information and propagate it downstream, thereby driving the circuit’s functional state. Furthermore, the interconnections between DFFs determine performance metrics such as arrival times. Therefore, by progressively and accurately modeling each DFF and its immediate connectivity network, we can localize the critical information within shorter-range dependencies. This approach reduces the reliance on long-range information propagation. However, existing methodologies have not yet taken advantage of this aspect for targeted modeling.

On the other hand, compared to the vast number of cell-level nodes present in the netlist, the RTL code offers a more macroscopic description of circuit connections and the functionality they form. As illustrated in Fig. 1(d), complex connectivity structures in a netlist can be succinctly represented by a single line of RTL code, providing a clear overview of the circuit’s functionality. Therefore, if we can simultaneously learn from both RTL code and circuit netlist structures, integrating local and global information collaboratively, it will significantly enhance the efficiency of circuit representation learning. Moreover, by utilizing the formal language of RTL code—which provides an effective abstraction of functionality—and an LLM fine-tuned on circuit data to interpret the RTL, we can generate more accurate functional descriptions.

In this paper, we propose MOSS, a multimodal framework that integrates GNN with LLMs. We fine-tune an LLM on RTL code to generate embeddings that capture global circuit functionality. These embeddings enhance the features of DFF cells in the netlist, enabling the GNN to focus on critical anchor points and reduce reliance on long-range dependencies. The LLM also provides a global functionality embedding, offering efficient supervision for functionality-related tasks. Additionally, we introduce an adaptive aggregation method and a two-stage propagation mechanism to better model signal propagation and sequential feedback within the circuit. By combining structural insights from GNNs with functional understanding from LLMs, MOSS improves the accuracy of functionality and performance predictions for sequential circuits, particularly in larger circuits where existing methods struggle.

The contributions of this paper are summarized as follows:

- We propose MOSS, a multimodal framework that combines LLMs with GNNs for sequential circuit modeling, enhancing the collaboration between local node embeddings and global representations for circuit functionality and structure.
- We introduce an adaptive aggregation method and a two-phase propagation mechanism in the GNN to better model signal propagation and sequential feedback within circuits.
- MOSS significantly improves the accuracy of functionality and performance predictions for sequential circuits, outperforming existing methods on larger circuits. For example, it achieves a 95.2% accuracy in arrival time prediction.

II. BACKGROUND AND RELATED WORK

A. Graph Neural Networks for EDA Tasks

Graph neural networks (GNNs) are a class of deep learning models specifically designed to process graph-structured data [15]. By operating directly on graphs, they capture complex dependencies and interactions between nodes. GNNs iteratively aggregate information from neighboring nodes to update node representations [16], enabling them to capture both local and global structural features [17]. In the field of EDA, GNNs are widely used for tasks such as circuit representation learning, performance prediction, and fault detection due to their high adaptability to circuit structures [18], [19].

For instance, FGNN2 [20] and DeepGate2 [21] focused on combinational circuits, which lack the timing dependencies inherent in sequential circuits. The DeepSeq series [14], [22] sought to incorporate temporal information into GNNs by training on logic and flip probabilities to model timing behavior. DeepSeq2 advanced this approach by introducing disentangled structure, function, and timing representations. However, as in Fig. 1, DeepSeq series struggles with long-range information dependencies. This shortcoming leads to a substantial decline in accuracy when applied to larger-scale circuits. Additionally, the DeepSeq series performs learning on And-Inverter Graphs (AIGs), which, compared to real netlists composed of standard cells, have nodes with simpler and uniform functions. In EDA scenarios, crucial labels such as timing are annotated at the standard cell level rather than the AIG level. Thus, AIG-based models cannot capture these metrics directly, limiting industrial applicability. In contrast, our work models netlists composed of standard cells.

B. Large Language Models and Fine-Tuning for EDA Tasks

Large language models (LLMs) have shown promise in various EDA tasks, such as design flow automation, error detection, and hardware generation [23]. Pre-trained on vast amounts of textual data, LLMs excel at understanding RTL code and descriptions [24]. Fine-tuning techniques like low-rank adaptation (LoRA) enable efficient adaptation of LLMs to EDA-specific tasks [25]. However, while LLMs can capture the functional semantics from RTL code, their ability to model the complex structural dependencies inherent in circuits is limited, as they were initially designed for processing textual data [26]. Therefore, LLMs alone are insufficient for circuit design tasks that require both functional understanding and detailed structural representation.

C. Multimodal Alignment and Contrastive Learning

Multimodal alignment and contrastive learning have been successfully employed to integrate diverse data types [27], such as text and images [28], by aligning their representations in a shared embedding space [29]. Prominent models like CLIP [30] demonstrate the potential of contrastive learning for cross-modal tasks, including retrieval and classification. These approaches leverage different modalities’ strengths to enhance the performance of representation learning [31].

III. PROBLEM FORMULATION

Sequential circuits, modeled as directed graphs $G = (V, E)$, where V represents circuit components (e.g., logic gates, DFFs) and E represents component connections, pose unique challenges due to the temporal dependencies introduced by state elements. The key problem is to learn node embeddings $\mathbf{H} \in \mathbb{R}^{|V| \times d}$ that encode both structural and temporal features. These embeddings should support downstream tasks such as toggle rate prediction, arrival time prediction, and circuit classification:

$$\mathbf{H} = f(G, \mathbf{X}, \mathbf{T}) = f(V, E, \mathbf{X}, \mathbf{T}), \quad (1)$$

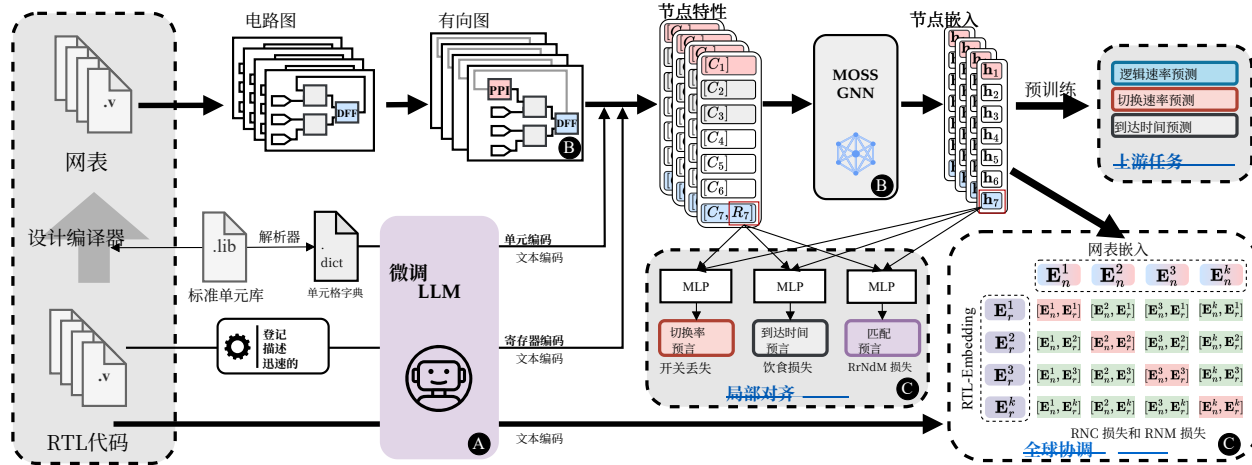


图 2 MOSS 框架概述。

其中 $\mathbf{X} \in \mathbb{R}^{|V| \times d_x}$ 表示结构特征, $\mathbf{T} \in \mathbb{R}^{|V| \times d_t}$ 表示时间特征。

电路表示学习的主要挑战是捕获顺序电路中的长程依赖关系和时间动态, 而当前大多数 GNN 都在与过度平滑作斗争, 无法保留关键的时间信息。集成多模式数据 (例如网表和 RTL 描述) 对于全面理解功能和结构方面至关重要。

四. 莫斯框架

图 2 描述了 MOSS 的框架。我们集成了 LLM 和 GNN, 以捕获 RTL 代码中的功能语义和网表中的结构依赖关系。我们的主要方法如下:

- LLM 增强的节点特征 (图 2A): LLM 在 RTL 数据上进行微调以生成上下文嵌入, 丰富了 GNN 的节点特征, 特别是寄存器节点 (DFF)。详细信息在第 IV-A 节中描述。
- 图构建和建模 (图2B): 网表表示为有向图。MOSS 使用多个聚合器和伪主输入 (PPI)/DFF 节点来对时序电路中的时间依赖性进行建模。详细信息在第 IV-B 节中描述。
- 局部和全局对齐 (图 2 C): MOSS 将 GNN 嵌入与 LLM 生成的 RTL 嵌入对齐。在本地, 诸如切换率和到达时间预测之类的任务使用 Etoggle Loss (增强型切换损失)、EAT Loss (增强型到达时间损失) 和 RrNdM (RTL 寄存器到 Netlist-DFF 匹配损失) 进行优化。在全局范围内, RNC 损失 (RTL-网表对比损失) 和 RNM 损失 (RTL-网表匹配损失) 对齐网表和 RTL 嵌入。详细信息在第 IV-C 节中描述。

A. LLM微调 and 节点功能增强

为了解决时序电路建模的挑战, 我们通过利用微调的 LLM 来生成 DFF 节点的上下文嵌入和每个逻辑单元的嵌入, 从而增强了 GNN 的节点特征。此外, 为了对电路功能任务提供更有效的训练监督, 我们使用 LLM 为完整的 RTL 代码生成嵌入。为此, 我们专门对 LLM 进行了微调, 以增强其对 RTL 代码的理解。

LLM微调。我们在 31,701 个 RTL 描述上对开源代码生成模型 Yi-Coder-9B-Chat [32], 进行了微调, 以提取寄存器和逻辑单元的上下文嵌入。

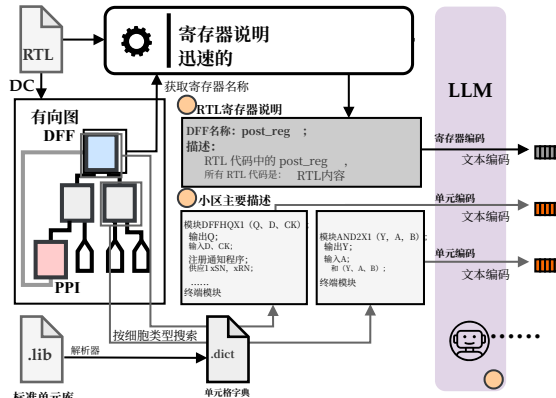


图3 GNN节点的LLM特征提取过程。

数据提取和 RTL 描述。为了增强电路节点的特征, 我们识别电路中的每个节点并提取其描述, 如图3A所示。我们首先使用设计编译器 (DC) [33]来综合网表, 从而产生有向图结构。然后我们识别 DFF 和逻辑单元的每个元素。此外, 对于网表中的每个节点, 我们提取其单元描述, 其中通常包括结构和功能信息。特别是对于DFF, 我们在RTL代码中找到相应的寄存器并生成寄存器描述提示。这些提示使法学硕士能够描述每个 DFF 的上下文和功能, 捕获本地和全局功能关系。

LLM-GNN 特征集成和对齐。使用提取的节点信息和描述, 我们使用 LLM 生成嵌入, 如图 3 B 所示。我们使用均值池来聚合来自 LLM 的标记嵌入, 提供输入序列的全面表示。LLM 生成的节点嵌入与 GNN 中每个节点的结构特征连接起来。对于 DFF, 生成的嵌入结合了 RTL 描述和 DFF 单元描述, 捕获结构和功能信息。对于逻辑单元, 嵌入对应于每个单元的单元描述。

B. 图构建和时间依赖建模

在我们的 GNN 设计中, 我们首先利用 MOSS 的多模态特性来生成节点嵌入。接下来, 我们设计了一个自适应聚合器, 以更好地适应标准单元级网表。最后,

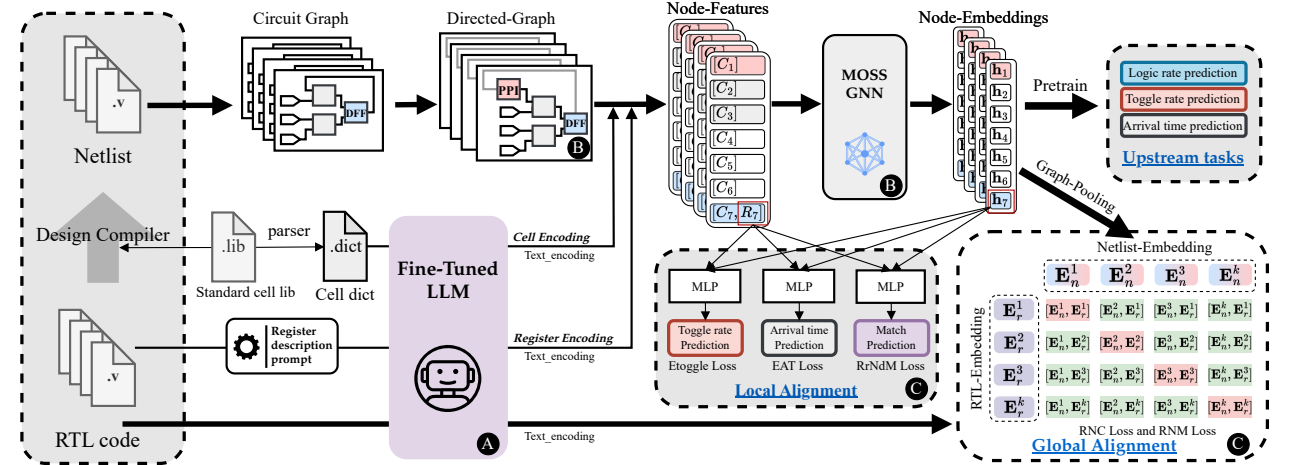


Fig. 2 Overview of the MOSS framework.

where $\mathbf{X} \in \mathbb{R}^{|V| \times d_x}$ represents structural features, and $\mathbf{T} \in \mathbb{R}^{|V| \times d_t}$ represents temporal features.

The main challenge of circuit representation learning is capturing long-range dependencies and temporal dynamics in sequential circuits, while most current GNNs struggle with over-smoothing and fail to retain critical temporal information. Integrating multimodal data, such as netlists and RTL descriptions, is crucial for comprehensively understanding functional and structural aspects.

IV. MOSS FRAMEWORK

Fig. 2 describes MOSS's framework. We integrate LLM and GNN to capture both functional semantics from RTL code and structural dependencies from netlists. Our key approaches are listed below:

- LLM-enhanced node features (Fig. 2 A): The LLM is fine-tuned on RTL data to generate contextual embeddings, enriching the GNN's node features, particularly for register nodes (DFFs). Details are described in Section IV-A.
- Graph construction and modeling (Fig. 2 B): The netlist is represented as a directed graph. MOSS uses multiple aggregators and Pseudo Primary Input (PPI)/DFF nodes to model temporal dependencies in sequential circuits. Details are described in Section IV-B.
- Local and global alignment (Fig. 2 C): MOSS aligns GNN embeddings with LLM-generated RTL embeddings. Locally, tasks like toggle rate and arrival time prediction are optimized using Etoggle Loss (enhanced toggle loss), EAT Loss (enhanced arrival time loss) and RrNdM (RTL-register to Netlist-DFF matching loss). Globally, RNC Loss (RTL-Netlist contrastive loss) and RNM Loss (RTL-Netlist matching loss) align the netlist and RTL embeddings. Details are described in Section IV-C.

A. LLM Fine-Tuning and Node Feature Enhancement

To address the challenges of modeling sequential circuits, we enhance the GNN's node features by leveraging a fine-tuned LLM to generate contextual embeddings for DFF nodes and embeddings for each logic cell. Additionally, to provide more effective training supervision for circuit functionality tasks, we use the LLM to generate embeddings for the complete RTL code. To this end, we specifically fine-tuned an LLM to enhance its understanding of RTL code.

LLM Fine-Tuning. We fine-tuned Yi-Coder-9B-Chat [32], an open-source code generation model, on 31,701 RTL descriptions to extract contextual embeddings for registers and logic cells.

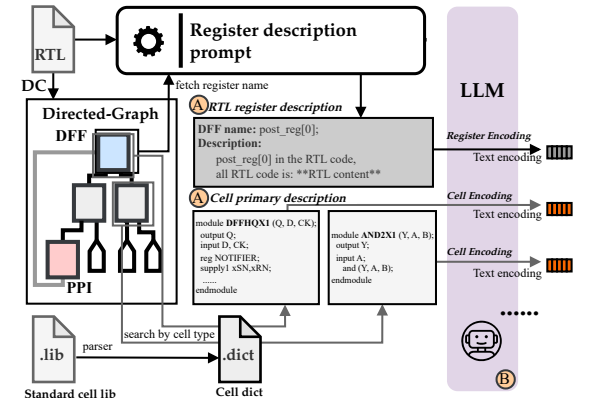


Fig. 3 LLM feature extraction process for GNN nodes.

Data Extraction and RTL Descriptions. To enhance the features of circuit nodes, we identify each node in the circuit and extract its description as shown in Fig. 3 A. We first use Design Compiler (DC) [33] to synthesize the netlist, resulting in a directed graph structure. We then identify each elements of DFFs and logic cells. Further, for each node in the netlist, we extract its cell description, which typically includes structural and functional information. Specifically for DFFs, we locate the corresponding registers in the RTL code and generate Register Description Prompts. These prompts enable the LLM to describe the context and functionality of each DFF, capturing both local and global functional relationships.

LLM-GNN Feature Integration and Alignment. Using the extracted node information and descriptions, we generate embeddings with the LLM as illustrated in Fig. 3 B. We use mean pooling to aggregate token embeddings from the LLM, providing a comprehensive representation of the input sequence. LLM-generated node embeddings are concatenated with each node's structural features in the GNN. For DFFs, the produced embeddings combine the RTL description and the DFF cell description, capturing both structural and functional information. For logic cells, the embeddings corresponds to each cell's cell description.

B. Graph Construction and Temporal Dependency Modeling

In our GNN design, we first utilize the multimodal characteristics of MOSS to generate node embeddings. Next, we design an adaptive aggregator to better accommodate standard cell-level netlists. Finally,

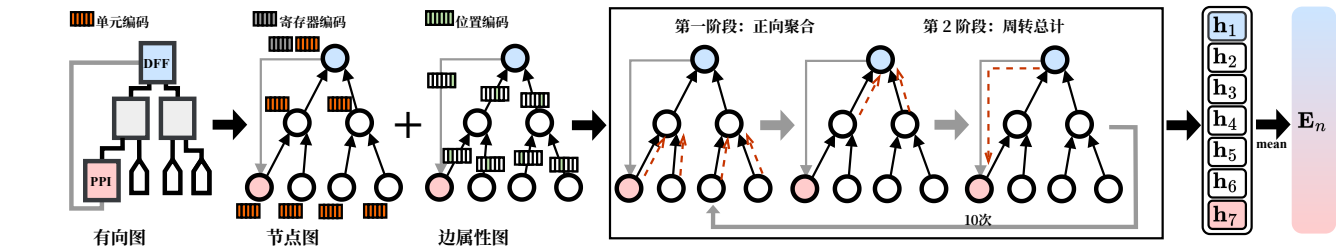


图4(a) MOSS中的图结构；（b）多编码两相异步时间传播；（c）均值池策略 f 图形表示。 or

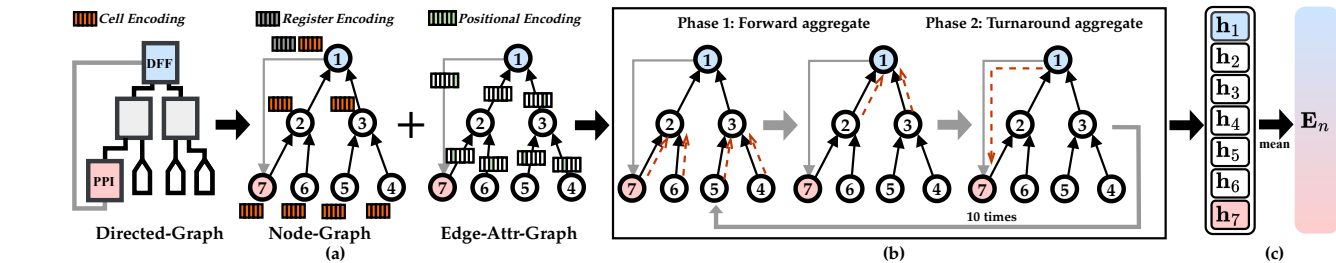


Fig. 4 (a) Graph structure in MOSS; (b) Two-phase Asynchronous temporal propagation with multi-encoding; (c) Mean-pooling strategy for graph representation.

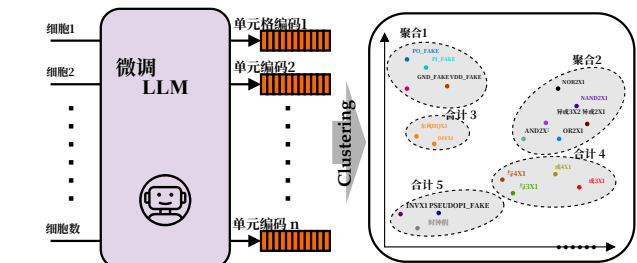


图 5 具有自适应聚类的自适应聚合器设计。

我们采用两相异步时间传播机制来更准确地模拟电路中的信号传播。

图构建。在构建图表时，我们与之之前的工作不同，之前的工作依赖于手动嵌入从电路图的结构信息中派生的标签。相反，我们利用 LLM 来处理 RTL 代码和单元节点的描述，从而利用 MOSS 的多模式功能。这种方法为图节点生成嵌入，如图 4(a) 所示。

首先，对于每个单元节点，我们利用 LLM 根据标准单元库中单元的功能和结构描述生成嵌入。具体来说，对于顺序电路结构（DFF）中的“锚点”，我们进一步增强了它们的嵌入。我们将 RTL 代码中相应寄存器节点的上下文结构和功能描述的 LLM 嵌入叠加到 DFF 节点上。这一增强功能有助于 GNN 在 DFF 节点上更准确地捕获更广泛的信息。

最后，由于单元的不同边缘具有不同的属性，例如，在某些逻辑门中，不同的输入对输出有不同的影响，我们对边缘进行位置编码。

自适应聚合器设计。由于我们在标准单元级别而不是现有设计中使用的 AIG 级别进行操作，因此我们的网表包含具有不同功能和结构的单元，并且不同的单元库可能包含不同的单元类型。因此，我们提出了基于聚类的自适应聚合器，用于标准单元级网表，以自动适应不同类型的单元。

如图 5 所示，MOSS 首先使用基于密度的噪声应用空间聚类 (DBSCAN) 和层次聚类，根据 LLM 派生的嵌入对节点进行动态分组。DBSCAN 基于功能相似性对节点进行聚类，检测不同密度的簇，而无需提前指定簇的数量。层次聚类通过考虑功能相似性和结构依赖性（例如扇入和扇出）来进一步细化这些簇。

然后，每个组都由专用聚合器处理，该聚合器学习根据节点的特征聚合信息。值得注意的是，我们采用图注意力机制来进行聚合

化，最终为每个类别形成不同的基于注意力的聚合器。这种自适应聚合器方法使 MOSS 能够灵活地适应不同的节点类型和交互，捕获对复杂电路建模至关重要的独特行为。

两相异步时间传播机制。为了准确地模拟时序电路中的时间依赖性，MOSS 采用了两阶段异步时间传播机制，如 [14] 所示。传播机制如图 4(b) 所示，包括前向和后向阶段来处理 DFF 的反馈行为。值得注意的是，两个阶段都使用异步更新来更类似于电路中的信号传播。

在第一阶段“前向传播”中，信号通过组合逻辑门从主输入 (PI) 传播到 DFF 的输入。此阶段捕获电路中的前向信号流，模拟输入信号如何影响下游组件。

在第二阶段（称为周转传播）中，DFF 的输出被反馈到电路中，模拟时序元件的特征反馈环路。这种反馈对于时序电路固有的时序依赖性和状态转换建模至关重要，使 GNN 能够捕获过去的状态如何影响未来的行为。为了捕获短程和长程依赖性，两阶段传播过程被重复多次（例如，10 次迭代）。

然后，如图 4 (c) 所示，使用均值池聚合最终节点表示，生成全面的嵌入，有效地总结电路中的时间和结构信息。

C. 本地和全球协调战略

我们的局部和全局对齐策略确保 GNN 生成的网表嵌入与 LLM 派生的 RTL 嵌入保持一致，从而捕获局部和全局电路特性。本节详细介绍了用于实现对齐的损失函数，重点关注切换率预测、到达时间预测和寄存器 DFF 匹配等关键任务。

局部对齐旨在通过特定于任务的损失来对齐节点，优化 GNN 表示局部电路行为的能力。局部对齐包含几个预测任务，它们的损失函数是 Etoggle 损失、EAT 损失和 RrNdM 损失。Etoggle Loss 最大限度地减少了预测和实际切换率之间的差异，捕获逻辑门的动态行为。EAT Loss 专注于预测信号到达时间，确保 GNN 准确地模拟时序依赖性。RrNdM Loss 可确保 RTL 寄存器和网表 DFF 之间的正确对齐，从而最大限度地减少寄存器到 DFF 映射中的不匹配。我们将这些损失函数实现为平滑 L1 损失函数。

全局对齐通过图确保网表和 RTL 嵌入的整体结构和功能之间的一致性

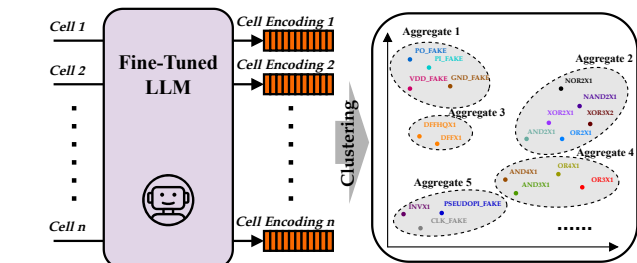


Fig. 5 Adaptive-aggregator design with adaptive clustering.

we employ a two-phase asynchronous temporal propagation mechanism to more accurately model signal propagation in circuits.

Graph Construction. In constructing the graph, we diverge from prior works that rely on manually embedding labels derived from the circuit graph’s structural information. Instead, we leverage the multimodal capabilities of MOSS by utilizing the LLM to process descriptions of the RTL code and cell nodes. This approach generates embeddings for the graph nodes, as illustrated in Fig. 4(a).

First, for each cell node, we utilize the LLM to generate embeddings based on the cell’s functional and structural descriptions in the standard cell library. Specifically, for the “anchor points” in sequential circuit structures, the DFFs, we further enhance their embeddings. We overlay the LLM embeddings of the contextual structure and functional descriptions of the corresponding register nodes from the RTL code onto the DFF nodes. This enhancement helps the GNN to capture a wider range of information more accurately at DFF nodes.

Finally, since different edges of cells have varying properties for example, in certain logic gates, different inputs have different influences on the output, we perform positional encoding on the edges.

Adaptive-Aggregator Design. Because we operate at the standard cell level rather than at the AIG level used in existing designs, our netlists contain cells with diverse functions and structures, and different cell libraries may include varying cell types. Therefore, we propose a clustering-based adaptive aggregator for standard cell-level netlists to adapt to different kinds of cells automatically.

As shown in Fig. 5, MOSS first uses Density-Based Spatial Clustering of Applications with Noise (DBSCAN) and hierarchical clustering to dynamically group nodes based on their LLM-derived embeddings. DBSCAN clusters nodes based on functional similarity, detecting clusters of varying density without specifying the number of clusters in advance. Hierarchical clustering further refines these clusters by considering both functional similarities and structural dependencies such as fan-in and fan-out.

Then, each group is processed by a dedicated aggregator that learns to aggregate information based on the characteristics of the nodes. Notably, we employ graph attention mechanisms for aggre-

gation, ultimately forming different attention-based aggregators for each category. This adaptive aggregator approach enables MOSS to flexibly adapt to different node types and interactions, capturing unique behaviors essential for modeling complex circuits.

Two-Phase Asynchronous Temporal Propagation Mechanism. To accurately model temporal dependencies in sequential circuits, MOSS adopts a two-phase asynchronous temporal propagation mechanism as in [14]. The propagation mechanism, illustrated in Fig. 4(b), includes forward and backward phases to handle the feedback behavior of DFFs. It is worth noting that both phases use asynchronous updates to more closely resemble signal propagation in circuits.

In the first phase, Forward Propagation, signals are propagated from primary inputs (PIs) to the inputs of DFFs through combinational logic gates. This phase captures the forward signal flow in the circuit, simulating how input signals affect downstream components.

In the second phase, referred to as Turnaround Propagation, the outputs from the DFFs are fed back into the circuit, simulating the characteristic feedback loops of sequential elements. This feedback is crucial for modeling the timing dependencies and state transitions intrinsic to sequential circuits, allowing the GNN to capture how past states influence future behavior. To capture both short-range and long-range dependencies, the two-phase propagation process is repeated multiple times (e.g., 10 iterations).

Afterward, as shown in Fig. 4(c), the final node representations are aggregated using mean-pooling, generating comprehensive embeddings that effectively summarize the temporal and structural information in the circuit.

C. Local and Global Alignment Strategy

Our local and global alignment strategy ensures that the GNN-generated netlist embeddings remain consistent with the LLM-derived RTL embeddings, capturing both local and global circuit characteristics. This section details the loss functions used to achieve alignment, focusing on key tasks such as toggle rate prediction, arrival time prediction, and register-DFF matching.

Local alignment aims to align nodes through task-specific losses, optimizing the GNN’s ability to represent local circuit behavior. The local alignment contains several prediction tasks, and their loss functions are Etoggle loss, EAT loss, and RrNdM loss. Etoggle Loss minimizes the difference between predicted and actual toggle rates, capturing logic gates’ dynamic behavior. EAT Loss focuses on predicting signal arrival times, ensuring the GNN accurately models timing dependencies. RrNdM Loss ensures proper alignment between RTL registers and netlist DFFs, minimizing mismatches in register-to-DFF mapping. We implement these loss functions as smooth L1 loss functions.

Global alignment ensures consistency between the overall structure and functionality of netlist and RTL embeddings through graph


```
# rtl_ 编码器 - 文本转换器
# 网表_编码器 - GNN
# R[n, l] - 对齐 RTL 代码的小批量
# N[n, c, m] - 对齐网表的小批量
# W_n[d_n, d_r] - 嵌入的投影
# MLP[2*d_r, 1] - 用于二元分类的 MLP
# t - 温度参数
# 提取每种模态的嵌入
R_f =rtl_编码器(R) # [n, d_r]
N_f =网表_编码器(N) # [n, d_n]
# 联合多模态嵌入 [n, d_r]
R_e = l2_归一化(R_f, axis=1)
N_e = l2_归一化(np.dot(N_f, W_n), 轴=1)
# RTL-网表对比 (RNC) 损失
RNCLogits = np.dot(R_e, N_e.T) * np.exp(t)
RNCLabels = np.arange(n)
损失_r = 交叉_熵_损失(logits, 标签, 轴=0)
损失_n = 交叉_熵_损失(logits, 标签, 轴=1)
RNCLoss = (损失_r + 损失_n) / 2
# RTL 网表匹配 (RNM) 损失
RNMLogits = [MLP(np.concatenate(R_e[i], N_e[j]))
              对于范围 (n) 中的i 对于范围 (n) 中的j]
RNMLabels = np.eye(n)
RNMLoss = SmoothL1Loss(RNMLogits, RNMLabels)
```

图 6 全局对齐损失函数的类似 Numpy 的伪代码。

池化和对齐损失。RNC 损失使全局结构保持一致，而 RNM 损失则强制网表和 RTL 之间的功能一致性。RNC和RNM计算方法如图6伪代码所示。

MOSS 采用多任务学习策略来泛化多个与电路相关的任务。通过优化特定任务损失函数的加权组合，MOSS 捕获局部节点级行为和全局电路范围模式。总损失计算公式为：

$$L_{\text{total}} = \sum_{i=1}^n \lambda_i L_{\text{task}_i}, \quad (2)$$

其中 λ_i 动态调整以平衡每个任务的贡献，确保没有单一任务主导学习过程。这种方法增强了模型在不同的时序电路任务中有效泛化的能力。

MOSS 采用双重对齐策略，将局部特定任务损失（Etoggle、EAT 和 RrNdM）与全局一致性损失（RNC 和 RNM）相结合。这种方法可以捕获细粒度电路动态和全局结构行为，从而提高模型的准确性和通用性，特别是对于复杂的时序电路。

五、实验结果

A. 实验设置

数据集。我们收集了 31,701 个 RTL 设计，并使用 Synopsys Design Compiler (DC) 综合它们，并应用多轮优化。对于每个 RTL，我们生成了几个不同的电路，尺寸范围从 100 到 5000 个单元不等，全部基于实际设计。从这个集合中，我们进一步选择了具有不同功能和显着变化的 RTL 子集作为我们的实验数据集。

型号变体。我们评估了几个 MOSS 变体，以了解关键组件的影响：“MOSS”是完整模型；“MOSS w/o A”是没有局部-全局对齐策略的完整模型；“MOSS w/o AA”包含LLM功能，但省略了自适应聚合器和对齐；“MOSS w/o FAA”是没有所有 LLM 功能增强、自适应聚合器设计和

对齐策略。此外，我们还实现了最先进的基于 GNN 的模型 DeepSeq2 [14]，进行比较。

评估指标。我们使用四个指标评估所有模型：每个 DFF 的到达时间预测 (ATP) 的准确性、每个单元的切换率预测 (TRP) 的准确性、电路的功率预测 (PP) 的准确性以及功能等效预测 (FEP) 的准确性。请注意，这里的预测精度是通过以下方式衡量的

$$\text{accuracy} = 1 - \text{mean relative error}. \quad (3)$$

FEP 测量正确识别功能等效的 RTL 网表对的比率。值越高表示性能越好。使用商业 EDA 工具收集地面实况数据。到达时间 (AT) 是通过使用 PrimePower [34] 和 Synopsys DC [33] 对 DFF 节点进行时序分析获得的。切换率源自使用随机输入进行超过 60,000 个周期的 VCS [35] 模拟，PrimePower 根据其切换率报告功率。这些基本事实确保了 MOSS 预测的准确基准。

平台配置。实验在配备 NVIDIA A100 GPU 的 8 个 GPU 系统上进行。我们在 PyTorch 中实现了 MOSS，并使用 Adam 优化器进行训练，学习率为 6×10^{-4} ，批量大小为 32。每个实验运行 45 个 epoch，根据验证性能提前停止以防止过度拟合。

B. 实验结果与分析

在本节中，我们评估 MOSS 在各种任务中的性能，并将其与其变体和基线模型 DeepSeq2 进行比较。实验重点关注到达时间预测、切换率预测、功率预测和功能等效性检查等关键任务。我们将结果列于表一和表二，并分析图7和图8所示的训练损失曲线。

到达时间预测 (ATP)。对于到达时间预测 (ATP)，MOSS 的平均准确率达到 95.2%，明显优于 DeepSeq2 (79.1%)。在较大的电路中，例如 mult 16x32 到 48，MOSS 保持高精度 (94.3%)，而 DeepSeq2 则下降至 57.6%。这凸显了 MOSS 处理远程依赖性的能力，这是大型电路中的一个已知挑战。对齐策略对此任务的影响有限，如 MOSS w/o A 的性能 (94.9%) 所示，该性能仍然很高，但略低于完整模型。

切换速率预测 (TRP)。对于切换率预测 (TRP)，MOSS 的准确率达到 87.5%，优于 DeepSeq2 (76.4%)。在 pipeline reg 等电路中，MOSS 达到 92.4%，而没有 FAA 的 MOSS 下降到 63.6%，这凸显了自适应聚合和 LLM 增强的重要性，这是捕获时序电路中动态行为的关键。

功率预测 (PP)。对于功效预测 (PP)，MOSS 的准确率先，为 96.3%，而 DeepSeq2 的准确率为 88.4%。LLM 增强型节点功能使 MOSS 能够更有效地对功耗模式进行建模，特别是在 wb 数据复用器等电路中，MOSS 达到 96.2%，明显高于不含 FAA 的 MOSS (82.6%)。MOSS 和 MOSS w/o A (95.1%) 之间较小的性能差距表明，与特征增强相比，对齐对于功率估计来说不太重要。

功能等价预测 (FEP)。我们选择了几个数据集，每组有 512 对 RTL 和网表，并计算了它们预测的平均正确率。在功能等价检查方面，MOSS 表现出色，平均准确率达到 93.7%，远超其他算法。

```
# rtl_encoder - Text Transformer
# netlist_encoder - GNN
# R[n, l] - minibatch of aligned RTL codes
# N[n, c, m] - minibatch of aligned netlists
# W_n[d_n, d_r] - projection of embedding
# MLP[2*d_r, 1] - MLP for binary classification
# t - temperature parameter
# extract embedding of each modality
R_f = rtl_encoder(R) # [n, d_r]
N_f = netlist_encoder(N) # [n, d_n]
# joint multimodal embedding [n, d_r]
R_e = l2_normalize(R_f, axis=1)
N_e = l2_normalize(np.dot(N_f, W_n), axis=1)
# RTL-Netlist Contrastive (RNC) Loss
RNCLogits = np.dot(R_e, N_e.T) * np.exp(t)
RNCLabels = np.arange(n)
loss_r = cross_entropy_loss(logits, labels, axis=0)
loss_n = cross_entropy_loss(logits, labels, axis=1)
RNCLoss = (loss_r + loss_n) / 2
# RTL-Netlist Matching (RNM) Loss
RNMLogits = [MLP(np.concatenate(R_e[i], N_e[j]))
              for i in range(n) for j in range(n)]
RNMLabels = np.eye(n)
RNMLoss = SmoothL1Loss(RNMLogits, RNMLabels)
```

Fig. 6 Numpy-like pseudocode for loss function of global alignment.

pooling and alignment losses. While RNC loss aligns the global structure, RNM loss enforces functional consistency between netlist and RTL. RNC and RNM calculation method is shown in the pseudocode in Fig. 6.

MOSS employs a multi-task learning strategy to generalize across multiple circuit-related tasks. By optimizing a weighted combination of task-specific loss functions, MOSS captures local node-level behaviors and global circuit-wide patterns. The overall loss is formulated as:

$$L_{\text{total}} = \sum_{i=1}^n \lambda_i L_{\text{task}_i}, \quad (2)$$

where λ_i dynamically adjusts to balance the contribution of each task, ensuring that no single task dominates the learning process. This approach enhances the model’s ability to generalize effectively across diverse sequential circuit tasks.

MOSS employs a dual alignment strategy that integrates local task-specific losses (Etoggle, EAT, and RrNdM) with global consistency losses (RNC and RNM). This approach captures both fine-grained circuit dynamics and global structural behavior, enhancing the model’s accuracy and generalizability, particularly for complex sequential circuits.

V. EXPERIMENTAL RESULTS

A. Experimental Settings

Datasets. We collected 31,701 RTL designs and synthesized them using Synopsys Design Compiler (DC), applying multiple rounds of optimization. For each RTL, we generated several distinct circuits, with sizes ranging from 100 to 5000 cells, all based on real-world designs. From this collection, we further selected a subset of RTLs with diverse functionalities and significant variations as our experimental dataset.

Model Variants. We evaluate several MOSS variants to understand the impact of critical components: “MOSS” is the full model; “MOSS w/o A” is the full model withoutht local-global alignment strategy; “MOSS w/o AA” includes LLM features but omits adaptive-aggregator and alignment; “MOSS w/o FAA” is the model without all the LLM feature enhancement, the adaptive-aggregator design, and

the alignment strategy. Besides, we implement DeepSeq2 [14], the state-of-the-art GNN-based model, for the comparison.

Evaluation Metrics. We assess all models using four metrics: the accuracy of arrival time prediction (ATP) for each DFF, the accuracy of toggle rate prediction (TRP) for each cell, the accuracy of power prediction (PP) for the circuit, and the accuracy of functional equivalence prediction (FEP). Note here the prediction accuracy is measured by

$$\text{accuracy} = 1 - \text{mean relative error}. \quad (3)$$

FEP measures the rate of correctly identifying functionally equivalent RTL-netlist pairs. Higher values indicate better performance. Ground truth data is collected using commercial EDA tools. Arrival Time (AT) is obtained via timing analysis on DFF nodes using PrimePower [34] and Synopsys DC [33]. Toggle Rate is derived from VCS [35] simulations over 60,000 cycles with random inputs, and power is reported by PrimePower based on their toggle rates. These ground truths ensure accurate benchmarking of MOSS’s predictions.

Platform Configuration. Experiments were performed on an 8-GPU system with NVIDIA A100 GPUs. We implemented MOSS in PyTorch and trained using the Adam optimizer, with a learning rate of 6×10^{-4} and a batch size of 32. Each experiment ran for 45 epochs, with early stopping based on validation performance to prevent overfitting.

B. Experimental Results and Analysis

In this section, we evaluate the performance of MOSS across various tasks, comparing it with its variants and the baseline model DeepSeq2. The experiments focus on critical tasks such as arrival time prediction, toggle rate prediction, power prediction, and functional equivalence checking. We list results in TABLE I and TABLE II, and analyze the training loss curves shown in Fig. 7 and Fig. 8.

Arrival Time Prediction (ATP). For arrival time prediction (ATP), MOSS achieves an average accuracy of 95.2%, significantly outperforming DeepSeq2 (79.1%). In larger circuits, such as *mult_16x32_to_48*, MOSS maintains high accuracy (94.3%), while DeepSeq2 drops to 57.6%. This highlights MOSS’s ability to handle long-range dependencies, a known challenge in larger circuits. The alignment strategy has a limited impact on this task, as demonstrated by the performance of MOSS w/o A (94.9%), which remains high but is slightly lower than the full model.

Toggle Rate Prediction (TRP). For toggle rate prediction (TRP), MOSS achieves 87.5% accuracy, outperforming DeepSeq2 (76.4%). In circuits such as *pipeline_reg*, MOSS reaches 92.4%, whereas MOSS w/o FAA drops to 63.6%, underscoring the importance of the adaptive aggregation and LLM enhancements, which are key for capturing dynamic behaviors in sequential circuits.

Power Prediction (PP). For power prediction (PP), MOSS leads with 96.3% accuracy, compared to DeepSeq2’s 88.4%. The LLM-enhanced node features allow MOSS to model power consumption patterns more effectively, particularly in circuits like *wb_data_mux*, where MOSS achieves 96.2%, significantly higher than MOSS w/o FAA (82.6%). The smaller performance gap between MOSS and MOSS w/o A (95.1%) indicates that alignment is less critical for power estimation compared to feature enhancement.

Functional Equivalence Prediction (FEP). We selected several datasets with 512 pairs of RTL and netlist in each set and calculated the average correctness of their predictions. In functional equivalence checking, MOSS excels with an average accuracy of 93.7%, far

表 I MOSS 框架变体的性能比较

电路	#细胞	深度序列2 [14]			MOSS 不带 FAA			MOSS 不含 AA			MOSS 不带 A			MOSS		
		ATP	TRP	PP	ATP	TRP	PP	ATP	TRP	PP	ATP	TRP	PP	ATP	TRP	PP
最大选择器	278	81.4	78.7	94.6	47.0	75.8	88.6	82.3	85.2	94.5	95.4	89.4	99.9	95.6	90.5	99.9
管道规则	610	77.6	83.6	91.4	52.2	63.6	63.4	80.5	88.3	90.2	94.2	92.1	94.1	94.5	92.4	94.6
prbs发生器	643	87.8	76.5	71.7	57.2	72.7	81.7	78.6	82.5	90.8	92.0	87.4	94.5	93.0	85.4	95.1
移位寄存器24	731	86.9	80.9	90.4	58.2	63.9	75.4	82.4	85.6	92.5	96.2	90.2	97.6	95.8	89.0	97.5
错误记录器	812	79.5	83.2	94.3	58.5	59.0	81.3	81.2	80.2	95.3	94.5	85.4	99.5	95.0	86.3	99.7
签名的mac	1306	66.4	77.3	95.6	26.9	56.1	73.6	76.5	78.4	88.6	93.8	83.8	92.3	94.5	85.3	94.1
WB数据复用器	1364	95.7	64.3	88.6	45.3	25.5	82.6	85.4	75.6	88.5	98.8	82.9	91.2	99.1	83.3	96.2
多16x32至48	4144	57.6	66.6	80.1	19.3	40.1	54.1	75.2	72.3	85.4	93.9	84.8	91.5	94.3	87.9	93.5
平均的	-	79.1	76.4	88.4	45.6	57.1	75.1	80.3	81.0	90.7	94.9	87.0	95.1	95.2	87.5	96.3

ATP：到达时间预测准确度 (%). TRP：切换率预测精度 (%). PP：功率预测精度 (%)。MOSS w/o FAA：没有功能增强、自适应聚合器和对齐的 MOSS。MOSS w/o AA：没有对齐和自适应聚合器的 MOSS。MOSS w/o A：没有对齐的 MOSS。

表 II 不同电路源上的 RTL 网表功能等效预测精度 (FEP)

电路	MOSS 不含美国联邦航空局	MOSS 不含 AA	MOSS 不含 A	MOSS
github_0	4.8	19.4	24.1	91.4
github_1	5.3	20.3	33.6	95.0
github_2	10.0	23.7	32.0	94.3
拥抱脸_0	7.9	16.4	19.5	94.1
拥抱脸_1	8.7	18.3	22.9	93.6
拥抱脸_2	14.1	21.1	27.5	93.5
平均的	8.5	19.9	26.6	93.7

粗体数字表示每个电路的最佳性能所有电路均来自公共存储库（GitHub 和 HuggingFace）

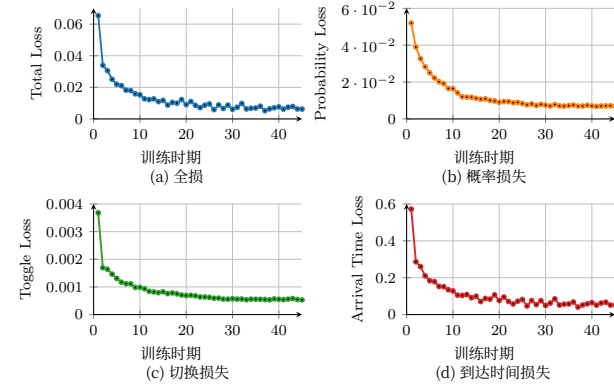


图 7 预训练部分的损失。

超过 MOSS w/o FAA (8.5%)、MOSS w/o AA (19.9%) 和 MOSS w/o A (26.6%)。变体的显着下降凸显了功能增强和对齐的必要性，以保持 RTL 和网表表示之间的功能一致性。例如，在 github_0 电路集中，MOSS 达到了 91.4%，而 MOSS w/o A 仅达到 24.1%，证实了多模态对齐的关键作用。该实验表明，通过利用多模态学习，MOSS 实现了卓越的电路功能建模，从而实现了功能等效性检查。

损失分析。如图7所示，在预训练阶段，所有损失分量稳步下降，总损失（见图7（a））显示出持续改善。概率损失（见图7（b））、切换损失（见图7（c））和到达时间损失（见图7（d））稳步下降，表明MOSS学会有效地捕获时序和动态行为。

图 8 显示了多模态对齐中的全局损失。总计

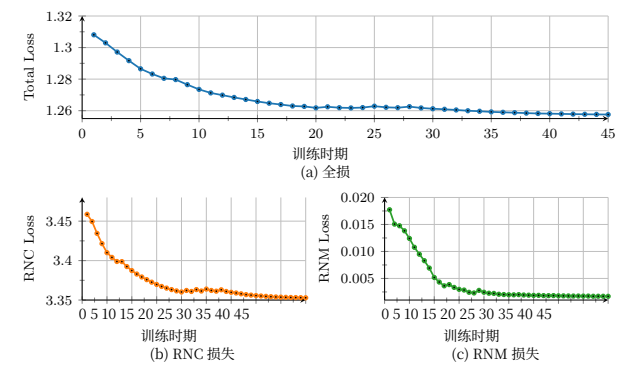


Fig. 8 多模式对齐中的全局损失行动。

45 个 epoch 后损失稳定在 1.26 左右，表明收敛有效。RNC损失（见图8（b））和RNM损失（见图8（c））也顺利收敛，证实了对齐策略的成功。RNM损失低至0.002，表明RTL网表匹配非常准确。

实验结果表明，MOSS 有效地解决了电路表示学习的挑战，特别是在处理具有远程依赖性的较大电路方面。通过集成 LLM 增强的特征和基于 GNN 的结构学习，MOSS 显著提高了到达时间、切换率和功率预测的准确性，同时通过多模态对齐确保功能一致性。

六. 结论

我们提出了 MOSS，一种结合 GNN 和 LLM 的多模态框架，以增强顺序电路表示。MOSS 通过利用 LLM 进行全局语义理解和 GNN 进行局部结构特征来解决远程信息丢失问题。它还引入了自适应聚合和两阶段传播来改进信号建模。实验结果表明，MOSS 显着提高了触发率和到达时间预测等任务的准确性，在大型电路的到达时间预测方面实现了 95.2% 的准确率，优于现有方法。我们希望MOSS不仅能为电路功能和性能预测提供有效的解决方案，还能激发更多的未来研究。

致谢

本文得到了中国科学院的部分资助（批准号：XDB0660102）和国家自然科学基金委员会（NSFC）的部分资助（批准号：62090024）。该论文的通讯作者为穆嘉楠和李华为。

TABLE I Performance Comparison of MOSS Framework Variants

Circuit	#Cells	DeepSeq2 [14]			MOSS w/o FAA			MOSS w/o AA			MOSS w/o A			MOSS		
		ATP	TRP	PP	ATP	TRP	PP	ATP	TRP	PP	ATP	TRP	PP	ATP	TRP	PP
max_selector	278	81.4	78.7	94.6	47.0	75.8	88.6	82.3	85.2	94.5	95.4	89.4	99.9	95.6	90.5	99.9
pipeline_reg	610	77.6	83.6	91.4	52.2	63.6	63.4	80.5	88.3	90.2	94.2	92.1	94.1	94.5	92.4	94.6
prbs_generator	643	87.8	76.5	71.7	57.2	72.7	81.7	78.6	82.5	90.8	92.0	87.4	94.5	93.0	85.4	95.1
shift_reg_24	731	86.9	80.9	90.4	58.2	63.9	75.4	82.4	85.6	92.5	96.2	90.2	97.6	95.8	89.0	97.5
error_logger	812	79.5	83.2	94.3	58.5	59.0	81.3	81.2	80.2	95.3	94.5	85.4	99.5	95.0	86.3	99.7
signed_mac	1306	66.4	77.3	95.6	26.9	56.1	73.6	76.5	78.4	88.6	93.8	83.8	92.3	94.5	85.3	94.1
wb_data_mux	1364	95.7	64.3	88.6	45.3	25.5	82.6	85.4	75.6	88.5	98.8	82.9	91.2	99.1	83.3	96.2
mult_16x32_to_48	4144	57.6	66.6	80.1	19.3	40.1	54.1	75.2	72.3	85.4	93.9	84.8	91.5	94.3	87.9	93.5
Average	-	79.1	76.4	88.4	45.6	57.1	75.1	80.3	81.0	90.7	94.9	87.0	95.1	95.2	87.5	96.3

ATP: Arrival Time Prediction accuracy (%). **TRP**: Toggle Rate Prediction accuracy (%). **PP**: Power Prediction accuracy (%). **MOSS w/o FAA**: MOSS without Feature Enhancement, Adaptive-Aggregator and Alignment. **MOSS w/o AA**: MOSS without Alignment and Adaptive-Aggregator. **MOSS w/o A**: MOSS without Alignment.

TABLE II RTL-netlist functional equivalence prediction accuracy (FEP) on different circuit sources

Circuit	MOSS w/o FAA	MOSS w/o AA	MOSS w/o A	MOSS
github_0	4.8	19.4	24.1	91.4
github_1	5.3	20.3	33.6	95.0
github_2	10.0	23.7	32.0	94.3
huggingface_0	7.9	16.4	19.5	94.1
huggingface_1	8.7	18.3	22.9	93.6
huggingface_2	14.1	21.1	27.5	93.5
Average	8.5	19.9	26.6	93.7

Bold numbers indicate the best performance for each circuit All circuits are from public repositories (GitHub and HuggingFace)

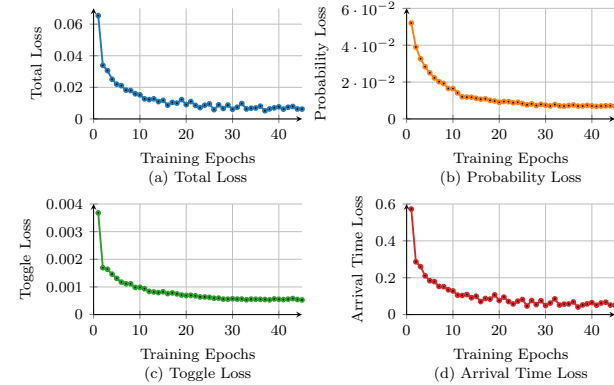


Fig. 7 Losses in the pre-training section.

exceeding MOSS w/o FAA (8.5%) , MOSS w/o AA (19.9%) and MOSS w/o A (26.6%). This significant drop in variants highlights the necessity of both feature enhancement and alignment to maintain functional consistency between RTL and netlist representations. For example, in the *github_0* circuit sets, MOSS achieves 91.4%, while MOSS w/o A only reaches 24.1%, confirming the critical role of multimodal alignment. This experiment shows that by leveraging multimodal learning, MOSS achieves superior circuit functionality modeling, enabling functional equivalence checking.

Loss Analysis. As illustrated in Fig. 7, in the pre-training phase, all loss components steadily decrease, with the total loss (see Fig. 7(a)) showing continuous improvement. Probability loss (see Fig. 7(b)), toggle loss (see Fig. 7(c)) and arrival time loss (see Fig. 7(d)) decrease steadily, indicating that MOSS learns to capture both timing and dynamic behaviors effectively.

Fig. 8 shows the global loss in multimodal alignment. The total

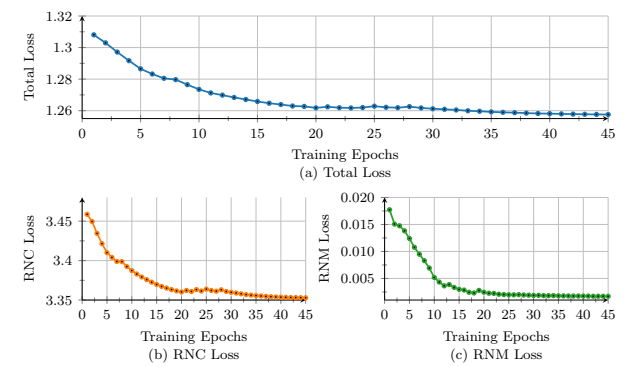


Fig. 8 Global Losses in the multimodal alignment section.

loss stabilizes around 1.26 after 45 epochs, indicating effective convergence. The RNC loss (see Fig. 8(b)) and RNM loss (see Fig. 8(c)) also converge smoothly, confirming the success of the alignment strategy. The RNM loss reaches as low as 0.002, indicating that the RTL-netlist matching is highly accurate.

The experimental results demonstrate that MOSS effectively addresses the challenges of circuit representation learning, particularly in handling larger circuits with long-range dependencies. By integrating LLM-enhanced features and GNN-based structural learning, MOSS significantly improves the accuracy of arrival time, toggle rate, and power predictions, while ensuring functional consistency through multimodal alignment.

VI. CONCLUSION

We propose MOSS, a multimodal framework combining GNNs and LLMs to enhance sequential circuit representation. MOSS addresses long-range information loss by leveraging LLMs for global semantic understanding and GNNs for local structural features. It also introduces adaptive aggregation and two-stage propagation to improve signal modeling. Experimental results show that MOSS significantly boosts accuracy for tasks like toggle rate and arrival time prediction, achieving 95.2% accuracy in arrival time prediction for large circuits, outperforming existing methods. We hope that MOSS can not just offer an effective solution for circuit functionality and performance prediction, but also stimulate more future research.

ACKNOWLEDGEMENT

This paper is supported in part by the Chinese Academy of Sciences under grant No. XDB0660102, and in part by the National Natural Science Foundation of China (NSFC) under grant No. (62090024). The corresponding authors of this paper are Jianan Mu and Huawei Li.

[1] L. 戴, B. Wang, X. Cheng, Q. Wang, X. Ni, “深度学习技术在集成电路设计中的应用”, 能源信息学, 第 1 卷. 7、没有. 1, 第 1-20 页, 2024 年. [2] C.田丽, 陈丽, 王Y, 王S, 周建, 庞Y, 杜Z, “基于机器学习的FPGA电子设计自动化技术综述”, 电子与信息学报, 2017年第1期. 45, 没有. 1, 第 1-13 页, 2023 年. [3] J. Yan, X. Lyu, R. Cheng 和 Y. Lin, “芯片设计中布局和布线的机器学习: 方法论概述”, arXiv 预印本 arXiv:2202.13564, 2022 年. [4] A. Al-Hyari, H. Szentimrey, A. Shamli, T. Martin, G. Grewal 和 S. Areibi, “预测 FPGA 电路布局可布线性的深度学习框架”, ACM Transactions on Reconfigurable Technology and Systems (TRETs), 卷. 14、没有. 3, 第 1-28 页, 2021 年. [5] R. Roy 和 S. Godil, “逻辑综合的机器学习”, 《电子设计自动化中的机器学习应用》. 施普林格, 2022 年, 第 183-204 页. [6] W. Haaswijk, E. Collins, B. Seguin, M. Soeken, F. Kaplan, S. S`usstrunk 和 G. De Micheli, “逻辑优化算法的深度学习”, IEEE 国际电路与系统研讨会 (ISCAS), 2018 年, 第 1-4 页. [7] A. Kumar, S. L. Tripathi 和 K. S. Rao, 《VLSI 芯片设计的机器学习技术》. 约翰·威利父子公司, 2023 年. [8] A. Choubey 和 S. B. Choubey, “用于测试 VLSI 电路的机器学习”, 《VLSI 和使用现代机器学习方法的硬件实现》. CRC Press, 2021 年, 第 23-40 页. [9] Z. 王成, 白, 何正, 张刚, 徐启, T.-Y. Ho, B. Yu 和 Y. Huang, “网表表示学习中的功能很重要”, ACM/IEEE 设计自动化会议 (DAC), 2022 年, 第 61-66 页. [10] P. Shrestha, S. Phatharodom 和 I. Savidis, “用于门到达时间预测的图形表示学习”, ACM/IEEE 机器学习 CAD (MLCAD) 研讨会, 2022 年, 第 127-133 页. [11] S. Liu, Y. Lu, W. Fang, M. Li 和 Z. Xie, “OpenLLM-RTL: LLM 辅助设计 RTL 生成的开放数据集和基准”, IEEE/ACM 国际计算机辅助设计会议 (IC-CAD), 2024 年. [12] C.-T. Ho 和 H. Ren, “用于标准单元布局设计优化的大型语言模型 (LLM)”, arXiv 预印本 arXiv:2406.06549, 2024 年. [13] Z. Dong, W. Cao, M. 张, D. Tao, Y. Chen 和 X. Zhang, “CktGNN: 用于电子设计自动化的电路图神经网络”, arXiv 预印本 arXiv: 2308.16406, 2023. [14] S. Khan, Z. Shi, Z. Cheng, M. Li 和 Q. Xu, “DeepSeq2: 通过解缠结表示增强序列电路学习”, arXiv 预印本 arXiv: 2411.00530, 2024. [15] J.周, G.崔, S.Hu, Z.Zhang, C.Yang, Z.Liu, L. Wang, C.Li, and M.Sun, “图神经网络: 方法和应用综述”, AI open, vol. 1, 第 57-81 页, 2020 年. [16] C.张, D.宋, C.黄, A.斯瓦米和N.V.Chawla, “异构图神经网络”, 第25届ACM SIGKDD知识发现和数据挖掘国际会议论文集, 2019年, 第793-803页. [17] Z. Ying, D. Bourgeois, J. You, M. Zitnik 和 J. Leskovec, “Gnnexplaner: 生成图神经网络的解释”, 神经信息处理系统进展, 卷. 2019 年 32 日. [18] E. Chien, M.Li, A.Aportela, K.Ding, S.Jia, S.Maji, Z.Zhao, J.Duarte, V.Fung, C.Hao 等, “图神经网络在电气工程中的机遇和挑战”, 《自然评论电气工程》, 卷. 1、没有. 8, 第 529–546 页, 2024 年. [19] A. Said, M. Shabbir, B. Broll, W. Abbas, P. V`olgyesi 和 X. Koutsoukos, “使用图神经网络完成电路设计”, 神经计算和应用, 卷. 35, 没有. 16, 第 12 145–12 157 页, 2023 年. [20] Z. 王成, 白, 何正, 张刚, 徐启, T.-Y. Ho, Y. Huang 和 B. Yu, “FGNN2: 用于学习电路逻辑功能的强大预训练框架”, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD), 2024 年. 史, H.潘, S.汗, M.李, Y.刘, J.黄, H.-L. Chen, M. Yuan, Z. Chu 和 Q. Xu, “DeepGate2: 功能感知电路表示学习”, IEEE/ACM 国际计算机辅助设计会议 (ICCAD), 2023 年, 第 1-9 页. [22] S. Khan, Z. Shi, M. Li 和 Q. Xu, “DeepSeq: 深度序列电路学习”, 发表于 IEEE/ACM 欧洲设计、自动化和测试论文集 (日期), 2024 年, 第 1-2 页.

[23] Z. 他和 B. Yu, “EDA 的大型语言模型: 未来还是幻影?” 2024 年国际物理设计研讨会论文集, 2024 年, 第 65-66 页. [24] A. Korvala, “优化和设计 VHDL 代码中的 LLM 模型分析”, 2023 年. [25] U. 夏尔马, B.-Y. Wu, S. R. D. Kankipati, V. A. Chhabria 和 A. Rovinski, “OpenROAD-Assistant: 用于物理设计任务的开源大型语言模型”, 载于 ACM/IEEE 机器学习 CAD (MLCAD) 研讨会, 2024 年, 第 1-7 页. [26] M. Abdollahi, S. F. Yeganli, M. A. Baharloo 和 A. Baniasadi, “大型语言模型的硬件设计和验证: 文献调查、挑战和未决问题”, 2024 年. Baltru`saitis, C. Ahuja 和 L.-P. Morency, “多模态机器学习: 调查和分类”, IEEE 模式分析和机器智能交易, 卷. 41、没有. 2, 第 423–443 页, 2018 年. [28] J. Li, D. Li, C. Xiong 和 S. Hoi, “Blip: 引导语言图像预训练以实现统一视觉语言理解和生成”, 国际机器学习会议. PMLR, 2022 年, 第 12 888–12 900 页. [29] A. Rahate, R. Walambe, S. Ramanna 和 K. Kotecha, “多模式协同学习: 挑战、数据集应用、最新进展和未来方向”, 信息融合, 卷. 81, 第 203–239 页, 2022 年. [30] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark 等人, “从自然语言监督中学习可转移视觉模型”, 国际机器学习会议 (ICML), 2021 年, 第 8748-8763 页. [31] P. H. Le-Khac, G. Healy 和 A. F. Smeaton, “对比表征学习: 框架和评论”, Ieee Access, 卷. 8, 第 193 907–193 934 页, 2020 年. [32] A. Young, B. Chen, C. Li, C. Huang, G. Zhu, G. Zhu, J. Chen, J. Chang 等人, “Yi: 01.ai 的开放基础模型”, arXiv 预印本 arXiv:2403.04652, 2024. [33] Synopsys, Inc., 设计编译器® 用户指南, Synopsys, Inc., 美国加利福尼亚州山景城, 2023 年 4 月. [34] Synopsys, Inc., “Primepower 用户指南”, Synopsys, Inc., 美国加利福尼亚州山景城, 2022 年 3 月. [35] Synopsys, Inc., VCS 用户指南, Synopsys, Inc., 美国加利福尼亚州山景城, 2019 年 9 月.

REFERENCES

[1] L. Dai, B. Wang, X. Cheng, Q. Wang, and X. Ni, “The application of deep learning technology in integrated circuit design,” *Energy Informatics*, vol. 7, no. 1, pp. 1–20, 2024. [2] C. Tian, L. Chen, Y. WANG, S. Wang, J. Zhou, Y. Pang, and Z. Du, “A survey on fpga electronic design automation technology based on machine learning,” *Journal of Electronics & Information Technology*, vol. 45, no. 1, pp. 1–13, 2023. [3] J. Yan, X. Lyu, R. Cheng, and Y. Lin, “Towards machine learning for placement and routing in chip design: a methodological overview,” *arXiv preprint arXiv:2202.13564*, 2022. [4] A. Al-Hyari, H. Szentimrey, A. Shamli, T. Martin, G. Grewal, and S. Areibi, “A deep learning framework to predict routability for FPGA circuit placement,” *ACM Transactions on Reconfigurable Technology and Systems (TRETs)*, vol. 14, no. 3, pp. 1–28, 2021. [5] R. Roy and S. Godil, “Machine Learning for Logic Synthesis,” in *Machine Learning Applications in Electronic Design Automation*. Springer, 2022, pp. 183–204. [6] W. Haaswijk, E. Collins, B. Seguin, M. Soeken, F. Kaplan, S. S`usstrunk, and G. De Micheli, “Deep learning for logic optimization algorithms,” in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2018, pp. 1–4. [7] A. Kumar, S. L. Tripathi, and K. S. Rao, *Machine Learning Techniques for VLSI Chip Design*. John Wiley & Sons, 2023. [8] A. Choubey and S. B. Choubey, “Machine Learning for Testing of VLSI Circuit,” in *VLSI and Hardware Implementations Using Modern Machine Learning Methods*. CRC Press, 2021, pp. 23–40. [9] Z. Wang, C. Bai, Z. He, G. Zhang, Q. Xu, T.-Y. Ho, B. Yu, and Y. Huang, “Functionality matters in netlist representation learning,” in *ACM/IEEE Design Automation Conference (DAC)*, 2022, pp. 61–66. [10] P. Shrestha, S. Phatharodom, and I. Savidis, “Graph representation learning for gate arrival time prediction,” in *ACM/IEEE Workshop on Machine Learning CAD (MLCAD)*, 2022, pp. 127–133. [11] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, “OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation,” in *IEEE/ACM International Conference on Computer-Aided Design (IC-CAD)*, 2024. [12] C.-T. Ho and H. Ren, “Large Language Model (LLM) for Standard Cell Layout Design Optimization,” *arXiv preprint arXiv:2406.06549*, 2024. [13] Z. Dong, W. Cao, M. Zhang, D. Tao, Y. Chen, and X. Zhang, “CktGNN: Circuit graph neural network for electronic design automation,” *arXiv preprint arXiv:2308.16406*, 2023. [14] S. Khan, Z. Shi, Z. Zheng, M. Li, and Q. Xu, “DeepSeq2: Enhanced Sequential Circuit Learning with Disentangled Representations,” *arXiv preprint arXiv:2411.00530*, 2024. [15] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, “Graph neural networks: A review of methods and applications,” *AI open*, vol. 1, pp. 57–81, 2020. [16] C. Zhang, D. Song, C. Huang, A. Swami, and N. V. Chawla, “Heterogeneous graph neural network,” in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 793–803. [17] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “Gnnexplainer: Generating explanations for graph neural networks,” *Advances in neural information processing systems*, vol. 32, 2019. [18] E. Chien, M. Li, A. Aportela, K. Ding, S. Jia, S. Maji, Z. Zhao, J. Duarte, V. Fung, C. Hao *et al.*, “Opportunities and challenges of graph neural networks in electrical engineering,” *Nature Reviews Electrical Engineering*, vol. 1, no. 8, pp. 529–546, 2024. [19] A. Said, M. Shabbir, B. Broll, W. Abbas, P. V`olgyesi, and X. Koutsoukos, “Circuit design completion using graph neural networks,” *Neural Computing and Applications*, vol. 35, no. 16, pp. 12 145–12 157, 2023. [20] Z. Wang, C. Bai, Z. He, G. Zhang, Q. Xu, T.-Y. Ho, Y. Huang, and B. Yu, “FGNN2: A Powerful Pre-Training Framework for Learning the Logic Functionality of Circuits,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 2024. [21] Z. Shi, H. Pan, S. Khan, M. Li, Y. Liu, J. Huang, H.-L. Zhen, M. Yuan, Z. Chu, and Q. Xu, “DeepGate2: Functionality-aware circuit representation learning,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023, pp. 1–9. [22] S. Khan, Z. Shi, M. Li, and Q. Xu, “DeepSeq: Deep Sequential Circuit Learning,” in *IEEE/ACM Proceedings Design, Automation and Test in Eurpoe (DATE)*, 2024, pp. 1–2.

[23] Z. He and B. Yu, “Large Language Models for EDA: Future or Mirage?” in *Proceedings of the 2024 International Symposium on Physical Design*, 2024, pp. 65–66. [24] A. Korvala, “Analysis of LLM-models in optimizing and designing VHDL code,” 2023. [25] U. Sharma, B.-Y. Wu, S. R. D. Kankipati, V. A. Chhabria, and A. Rovinski, “OpenROAD-Assistant: An Open-Source Large Language Model for Physical Design Tasks,” in *ACM/IEEE Workshop on Machine Learning CAD (MLCAD)*, 2024, pp. 1–7. [26] M. Abdollahi, S. F. Yeganli, M. A. Baharloo, and A. Baniasadi, “Hardware Design and Verification with Large Language Models: A Literature Survey, Challenges, and Open Issues,” 2024. [27] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “Multimodal machine learning: A survey and taxonomy,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 41, no. 2, pp. 423–443, 2018. [28] J. Li, D. Li, C. Xiong, and S. Hoi, “Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation,” in *International conference on machine learning*. PMLR, 2022, pp. 12 888–12 900. [29] A. Rahate, R. Walambe, S. Ramanna, and K. Kotecha, “Multimodal co-learning: Challenges, applications with datasets, recent advances and future directions,” *Information Fusion*, vol. 81, pp. 203–239, 2022. [30] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763. [31] P. H. Le-Khac, G. Healy, and A. F. Smeaton, “Contrastive representation learning: A framework and review,” *Ieee Access*, vol. 8, pp. 193 907–193 934, 2020. [32] A. Young, B. Chen, C. Li, C. Huang, G. Zhang, G. Zhang, H. Li, J. Zhu, J. Chen, J. Chang *et al.*, “Yi: Open foundation models by 01. ai,” *arXiv preprint arXiv:2403.04652*, 2024. [33] Synopsys, Inc., *Design Compiler® User Guide*, Synopsys, Inc., Mountain View, CA, USA, Apr. 2023. [34] Synopsys , Inc., “Primepower user guide,” Synopsys, Inc., Mountain View, CA, USA, Mar. 2022. [35] Synopsys, Inc., *VCS User Guide*, Synopsys, Inc., Mountain View, CA, USA, Sep. 2019.